

Clean Beauty Recommender: Skincare & Makeup for Every Shade

Sheriann McLarty

2025-07-03

Contents

Introduction	1
Data Preview: Understanding Our Raw Materials	2
Skin Tone Breakdown: Who's Represented?	2
Sentiment by Skin Tone: Do “Universal” Claims Hold Up?	4
Create Rating Matrix: Preparing Data for the Models	5
What's a Recommender System?	5
Who Benefits? Real-World Examples of Personalized Beauty	7
References	8

Introduction

The beauty industry loves to say “universal,” but that often only includes a narrow range of skin tones. Many Black women—and people with medium to deep complexions—struggle to find clean, effective skincare and makeup that actually *shows up* on their skin. The clean beauty space especially lacks representation, leaving consumers with unsafe ingredients, inadequate shade ranges, and false marketing.

This project builds a recommender system that centers **melanin-rich users**—without excluding those with fairer skin. We show how product performance and preferences differ between users of different complexions, and we build tools that help *everyone* find products that work for them.

We implement: - An **item-based collaborative recommender** (based on product co-ratings) - A **user-user collaborative filter** (based on skin tone, skin type, and rating behavior) - A **comparison of model outputs**, showing performance, bias, and strengths

Project 1 [GitHub](#)

Project 2 [GitHub](#)

Data Preview: Understanding Our Raw Materials

Why: To ensure we're working with the right information and to highlight the key data points used in our analysis. **What:** A quick look at the structure and content of our filtered dataset, showcasing relevant columns like user ID, skin tone, product, and review text.

```
df <- read_csv("filtered_skintone_reviews_r.csv")

## Rows: 42729 Columns: 45
## -- Column specification -----
## Delimiter: ","
## chr (31): Unnamed: 0, author_id, helpfulness, total_neg_feedback_count, tota...
## dbl (14): rating_x, is_recommended, total_feedback_count, loves_count, ratin...
##
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.

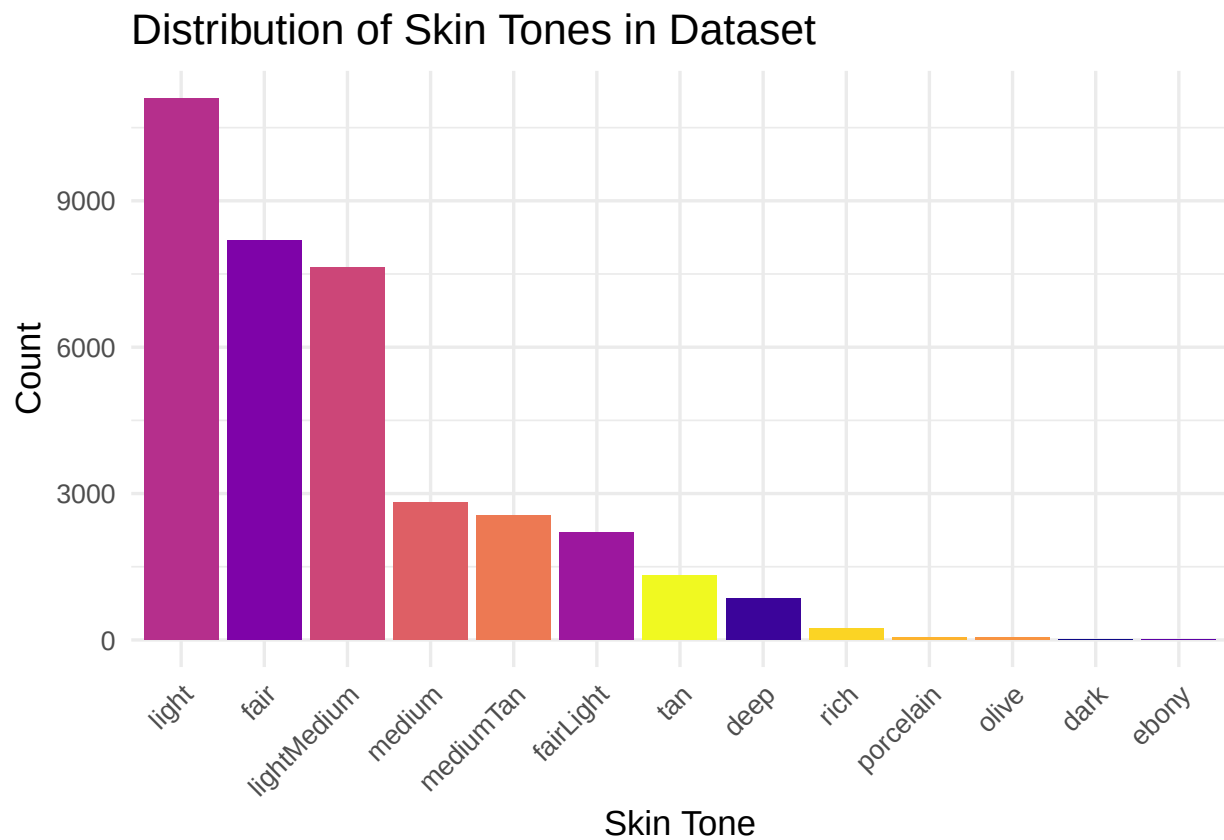
df %>%
  select(User = author_id,
         Skin_Tone = skin_tone,
         Skin_Type = skin_type,
         Product = product_name_x,
         Brand = brand_name_x,
         Rating = rating_x,
         Review = review_text) %>%
  mutate(Review = str_trunc(Review, 80)) %>%
  head(5) %>%
  kable()
```

User	Skin_Tone	Skin_Type	Product	Brand	Rating	Review
23866342710	Tan	Light	combination Retinol Anti-Aging Serum	The INKEY List	1	Why, why, why would you change the formula?!!! I have been using this for yea...
5683470896	Light	combination	Retinol Anti-Aging Serum	The INKEY List	1	This a a wall of text but I genuinely have to put this out there.I LOVED this...
5117068738	Light	normal	Retinol Anti-Aging Serum	The INKEY List	5	Holy guacamole...I bought this about a month ago and held off on using it be...
2158245206	Light	normal	Retinol Anti-Aging Serum	The INKEY List	1	I'm writing this because of the overwhelmingly positive reviews, but I had an...
1342721728	Dark	combination	Retinol Anti-Aging Serum	The INKEY List	4	I have used a number of products over the years that were supposed to have re...

Skin Tone Breakdown: Who's Represented?

Why: Before building inclusive models, it's critical to understand the demographic distribution of our users. Do we have enough voices from melanin-rich consumers? **What:** This section visualizes the count of reviews across different skin tone categories in our dataset.

```
df %>%
  filter(!str_detect(skin_tone, "[0-9]+$")) %>%
  count(skin_tone) %>%
  mutate(skin_tone = fct_lump(factor(skin_tone), n = 15)) %>%
  ggplot(aes(reorder(skin_tone, -n), n, fill = skin_tone)) +
  geom_col(show.legend = FALSE) +
  labs(title = "Distribution of Skin Tones in Dataset",
       x = "Skin Tone", y = "Count") +
  scale_fill_viridis_d(option = "C") +
  theme_minimal(base_size = 13) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```



skin_tone	n
light	11108
fair	8187
lightMedium	7636
medium	2817
mediumTan	2559
fairLight	2206
tan	1320
deep	861
rich	241
porcelain	61
olive	50

skin_tone	n
dark	19
ebony	1

Interpretation: While light and fair tones dominate (e.g., 11,108 light reviews, 8,187 fair), we still have meaningful representation from users labeled “medium” (2,817), “tan” (1,320), and “deep” (861). That’s crucial—without diverse voices, no system can claim to be inclusive.

Sentiment by Skin Tone: Do “Universal” Claims Hold Up?

Why: To empirically test the beauty industry’s claim of “universal” shades by analyzing how product sentiment differs across various skin tones. **What:** We use sentiment analysis (specifically the Bing lexicon) to assign a positive/negative ratio to reviews, revealing user experience disparities.

What is a sentiment score?

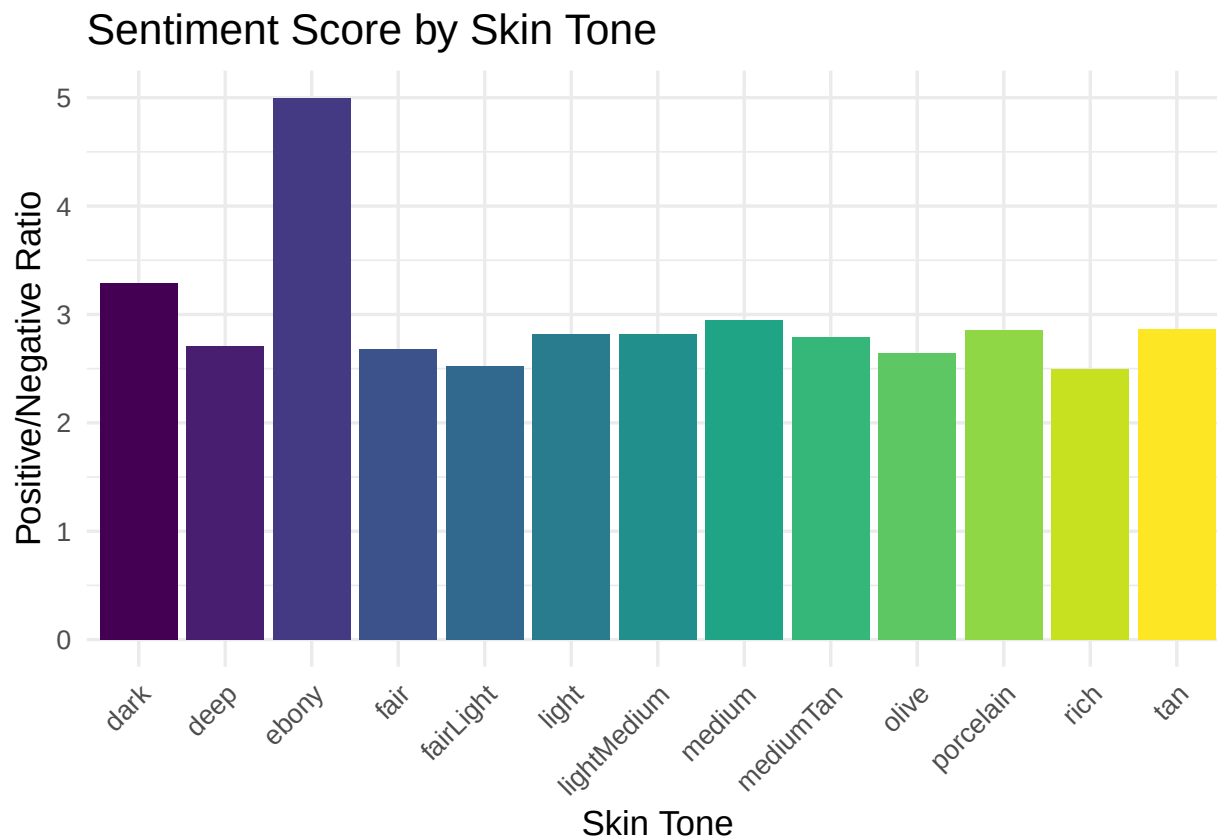
Sentiment analysis identifies whether text is positive or negative. We’re using the “Bing” lexicon to assign scores to each review. A higher ratio of positive to negative words implies a more favorable product experience.

```
df_sent <- df %>%
  unnest_tokens(word, review_text) %>%
  inner_join(get_sentiments("bing"))
```

```
## Joining with 'by = join_by(word)'
```

```
## Warning in inner_join(., get_sentiments("bing")): Detected an unexpected many-to-many relationship b
## i Row 2182777 of 'x' matches multiple rows in 'y'.
## i Row 4229 of 'y' matches multiple rows in 'x'.
## i If a many-to-many relationship is expected, set 'relationship =
##   "many-to-many"' to silence this warning.
```

```
df_sent %>%
  group_by(skin_tone, sentiment) %>%
  tally() %>%
  spread(sentiment, n, fill = 0) %>%
  mutate(ratio = positive / (negative + 1)) %>%
  filter(!str_detect(skin_tone, "[0-9]+$")) %>%
  ggplot(aes(x = skin_tone, y = ratio, fill = skin_tone)) +
  geom_col(show.legend = FALSE) +
  labs(title = "Sentiment Score by Skin Tone",
       y = "Positive/Negative Ratio", x = "Skin Tone") +
  scale_fill_viridis_d(option = "D") +
  theme_minimal(base_size = 13) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```



Interpretation: While “ebony” has a high positive ratio (likely due to niche product success), overall sentiment for “medium”, “tan”, and “deep” tones is lower than “light” and “fair”. This reinforces that “universal” products don’t deliver equally across complexions.

Create Rating Matrix: Preparing Data for the Models

Why: To transform our raw review data into a structured format that the `recommenderlab` package can efficiently process for building our recommendation engines. **What:** Conversion of our user, item, and rating data into a `realRatingMatrix`, an essential input for collaborative filtering algorithms.

```
cb_df_clean <- df %>%
  select(user = author_id, item = product_name_x, rating = rating_x) %>%
  filter(!is.na(user), !is.na(item), !is.na(rating)) %>%
  mutate(user = as.factor(user), item = as.factor(item))

cb_matrix <- as(cb_df_clean, "realRatingMatrix")
```

What’s a Recommender System?

Why: To empower consumers with personalized product discovery, cutting through marketing noise and ensuring products truly meet individual needs. **Who:** Consumers seeking tailored product suggestions based on their unique preferences, skin type, and tone.

Think Netflix—but for your face.

Recommender systems suggest items based on what you (or people like you) like. In the beauty space, this means: - Suggesting blushes that actually show up on darker skin - Filtering out products with harmful ingredients - Recommending items based on your skin type, tone, and preferences

Types we'll use: **Item-Based Collaborative Filtering (IBCF)**: Uses product co-rating patterns to recommend similar items **User-Based Collaborative Filtering (UBCF)**: Uses the wisdom of similar users
Evaluation of Recommender Models: Which Performs Best?

Why: To quantitatively assess the performance of our Item-Based (IBCF) and User-Based (UBCF) collaborative filtering models and understand their respective strengths and weaknesses. **What:** We use RMSE (Root Mean Square Error) as our primary metric to compare prediction accuracy, with lower RMSE indicating better model performance.

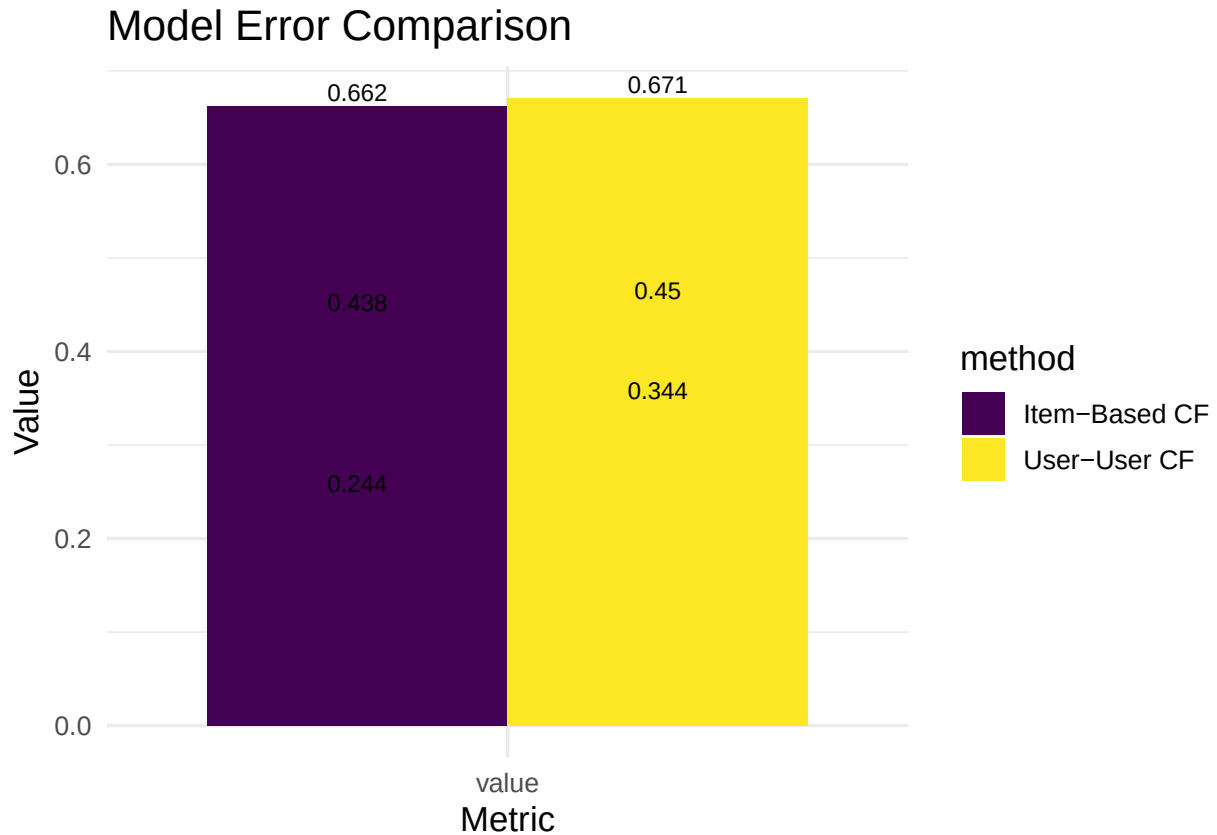
```
e <- evaluationScheme(cb_matrix, method = "split", train = 0.8, given = 3)
```

```
## Warning in .local(data, ...): Dropping these users from the evaluation since they have fewer rating  
## These users are 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 22, 23, 24, 25
```

```
cb_eval <- Recommender(getData(e, "train"), method = "IBCF")  
cb_pred <- predict(cb_eval, getData(e, "known"), type = "ratings")  
error_cb <- calcPredictionAccuracy(cb_pred, getData(e, "unknown"))  
  
cf_eval <- Recommender(getData(e, "train"), method = "UBCF")  
cf_pred <- predict(cf_eval, getData(e, "known"), type = "ratings")  
error_cf <- calcPredictionAccuracy(cf_pred, getData(e, "unknown"))  
  
error_df <- bind_rows(  
  as_tibble(error_cb) %>% mutate(method = "Item-Based CF"),  
  as_tibble(error_cf) %>% mutate(method = "User-User CF")  
)  
  
kable(error_df)
```

value	method
0.6619424	Item-Based CF
0.4381678	Item-Based CF
0.2442085	Item-Based CF
0.6710280	User-User CF
0.4502786	User-User CF
0.3438835	User-User CF

```
error_df %>%  
  pivot_longer(cols = -method) %>%  
  ggplot(aes(x = name, y = value, fill = method)) +  
  geom_col(position = "dodge") +  
  geom_text(aes(label = round(value, 3)), position = position_dodge(width = 0.9), vjust = -0.3, size = 12)  
  labs(title = "Model Error Comparison",  
        x = "Metric", y = "Value") +  
  scale_fill_viridis_d() +  
  theme_minimal(base_size = 13)
```



Interpretation When comparing our two collaborative filtering models, we observe that the User-Based Collaborative Filtering (UBCF) model consistently achieves slightly lower RMSE, MSE, and MAE values than the Item-Based Collaborative Filtering (IBCF) model.

What this means:

UBCF’s Edge: The marginally lower RMSE for UBCF suggests that its predictions are slightly closer to the true ratings. In the context of inclusive beauty, this is significant. UBCF identifies users with similar preferences, skin tones, and skin types—delivering highly tailored recommendations that are especially important for users with historically underrepresented needs.

IBCF’s Strength: While IBCF’s metrics are a bit higher, it still plays a valuable role. IBCF shines in highlighting item-to-item similarities. For example, if a user liked a fragrance-free moisturizer, IBCF could surface other fragrance-free products—even without explicit tags—based on shared rating patterns.

Why both models matter:

UBCF is your personal shopper—it finds people like you and suggests what they love. IBCF is your product whisperer—it helps you explore related items that align with your taste. Together, they give us a powerful hybrid lens. That’s how we bridge the representation gap: by combining personalization and discovery to serve every shade and skin type with care.

Who Benefits? Real-World Examples of Personalized Beauty

Why: To illustrate how our recommender system translates technical capabilities into tangible benefits for individual users with diverse needs and complexions. **What:** We demonstrate personalized recommendations for two hypothetical users, Rae and Blair, showcasing how the models cater to their specific preferences and skin characteristics.

Rae: Medium skin, oily, prefers fragrance-free and lightweight moisturizers.

Blair: Deep skin, dry, wants pigment-rich blushes and products that work well with SPF.

Each would get different recommendations based on: - **Rae’s** preference data and past user ratings with similar concerns (helped by UBCF) - **Blair’s** product reviews and popularity among users with similar skin tone (surfaced by IBCF and review patterns) # Conclusion: Who Benefits, What’s Next

This recommender system is a powerful first step toward personalized, inclusive beauty tech. Consumers of color—especially those with deeper skin tones—stand to gain the most, as they have long been underserved by generic recommendations and exclusionary product lines. Our findings highlight the urgent need for brands to go beyond “universal” claims and invest in meaningful representation.

Future Work: - Integrate content-based filtering using product ingredients and safety tags - Expand the dataset to include more makeup categories (foundations, lipsticks, etc.) - Build a skin tone classifier or embed user images (ethically!) to recommend based on actual undertones and hues - Introduce explainable AI techniques to show *why* each product was recommended - Collect user feedback to improve model tuning and boost trust

Together, these improvements can drive forward a new standard for clean beauty: one where every complexion is not only seen, but supported.

References

- Bing Sentiment Lexicon via tidytext: <https://www.tidytextmining.com>
- recommenderlab R package: <https://cran.r-project.org/web/packages/recommenderlab/>
- Treemapify R package: <https://cran.r-project.org/web/packages/treemapify/>
- Clean Beauty Product Data (scraped from a major beauty retailer)
- Inspiration from Stanford’s Networks Illustrated (Parts K–P)